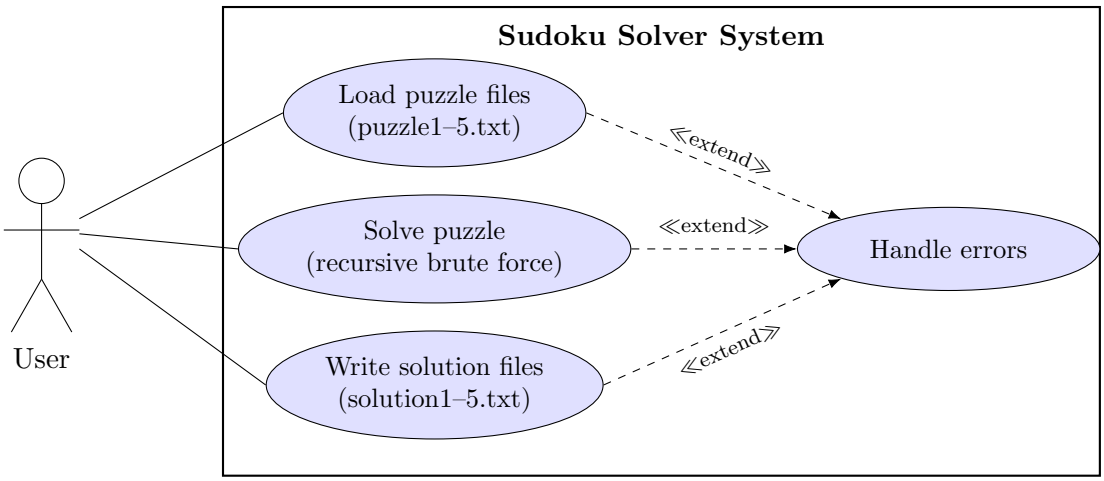
# EECS 348 Assignment 5: Use Case Diagram

Original by Christopher Warren
Revised by Nicolas Villalobos (with Claude, Anthropic)

October 29, 2026




## 1 Use Cases in the System

a. Load puzzle files

b. Solve puzzle

c. Write solution files (replaces "View all solutions")

d. Handle errors (optional, extends a, b, and c)

## 2 Brief Descriptions

**Load puzzle files.** The system no longer asks the user for a file name. When run, it automatically opens `puzzle1.txt` through `puzzle5.txt` from the same folder. For each one, it creates a new `SudokuSolver` object and reads the 81 characters (1–9 or _) into its $9 \times 9$ board.

**Solve puzzle.** The solver checks that the starting board breaks no rules, finds the first empty square, and recursively tries every valid digit (no repeat in the same row, column, or $3 \times 3$ box). It backtracks to find every solution, not just the first.

**Write solution files.** Each solution is written to `solutionN.txt` as soon as it is found, in the same format as the puzzle file. After solving, the file ends with `Solutions found: n`, or contains only `No solution found.` if there were none.

**Handle errors (optional).** If a puzzle file cannot be opened, has fewer than 81 entries, or contains an invalid character, the system prints an error and skips to the next puzzle. If the starting board breaks the rules, the solver is skipped and the file says `No solution found.`

## 3 Outline (Step-by-Step Flow)

**Load puzzle files**

1. User runs the executable.
2. System sets the puzzle number N to 1.
3. System builds the names `puzzleN.txt` and `solutionN.txt`.
4. System creates a new `SudokuSolver` object.
5. System opens a read stream for `puzzleN.txt`.
6. System reads 81 characters into the $9 \times 9$ board, checking each is 1–9 or _.
7. System closes the read stream.
8. System passes control to Solve puzzle.
9. System repeats steps 3–8 for N = 2 through 5.

*Extensions:*

a. File cannot be opened: system prints an error and skips to the next puzzle.

b. File has fewer than 81 entries: system prints that it expects a $9 \times 9$ grid and skips to the next puzzle.

c. File has an invalid character: system prints the character and skips to the next puzzle.

**Solve puzzle**

1. System opens `solutionN.txt` for writing and sets the solution count to 0.
2. System checks every given digit against the rules.
3. System finds the first empty square (left to right, top to bottom).
4. System tries digits 1–9 in that square, keeping only valid ones.
5. For a valid digit, system places it and finds the next empty square.
6. System calls itself recursively on that square.
7. When no empty square remains, system passes the full board to Write solution files.
8. System clears the square (backtracks) and tries the next digit.
9. Solving ends when the first call has tried all digits.

*Extensions:*

a. Starting board breaks the rules: system skips solving and goes to step 3 of Write solution files.

b. Board has no empty squares: system records it as one solution.

**Write solution files**

1. System increases the solution count by one.
2. System writes the board to `solutionN.txt`: nine lines, digits one space apart, then a blank line.
3. After solving ends, system writes `Solutions found: n`.
4. System closes `solutionN.txt` and prints `Wrote solutionN.txt` to the terminal.

*Extensions:*

a. Solution count is 0: system writes only `No solution found.`

b. `solutionN.txt` cannot be created: system prints an error and moves to the next puzzle.

# 4 Detailed Descriptions

**Load puzzle files**

- Triggered automatically when the user runs the program; there is no input prompt.
- `main` loops N from 1 to 5 and creates a fresh `SudokuSolver` for each puzzle, so no board or count carries over.
- `loadBoard` opens an `std::ifstream` and reads 81 non-whitespace characters into the board, ignoring spaces and newlines.
- Each character must be 1–9 or _; otherwise loading fails.
- On success, the stream is closed and the board is ready for Solve puzzle.

**Solve puzzle**

- `solveToFile` opens the output file, resets the count, and calls `validboard` to confirm no given digit repeats in its row, column, or box.
- It finds the first empty square and calls the recursive `solveSudoku`.
- `solveSudoku` tries digits 1–9 using `isValid`. For each valid digit it places it, scans forward for the next empty square, and recurses.
- After each recursive call, the square is reset to _ so other digits can be tried. This continues until every combination has been explored.

**Write solution files**

- Each time the board fills completely, the count increases and `printBoard` writes it to the `solutionN.txt` stream right away, so solutions are never stored in memory.
- The format matches the puzzle files: nine rows of digits separated by single spaces, followed by a blank line between solutions.
- After solving, the file ends with `Solutions found: n`, or contains only `No solution found.` if the count is 0.

**Handle errors (optional)**

- Unopenable files, short files, and invalid characters are reported on the terminal, and the program continues with the next puzzle instead of terminating.
- An invalid starting board is not an error that stops the program; it simply produces `No solution found.`